

Practical No.1

Aim:

Implement Caesar Cipher encryption and decryption.

Code:

```
interface EncryptParameters {
    plainText: string
    key: number
}

interface DecryptParameters {
    encryptedText: string
    key: number
}

const LOWERCASE_A = 97
const LOWERCASE_Z = 122
const ALPHABET_SIZE = 26

/** Wraps a char code into the lowercase a-z range (97-122). */
function wrapLowercaseCode(code: number): number {
    while (code > LOWERCASE_Z) code -= ALPHABET_SIZE
    while (code < LOWERCASE_A) code += ALPHABET_SIZE
    return code
}

/** Normalizes key to 0-25 so any integer key works. */
function normalizeKey(key: number): number {
    return ((key % ALPHABET_SIZE) + ALPHABET_SIZE) % ALPHABET_SIZE
}

function isLowercaseLetter(code: number): boolean {
    return code >= LOWERCASE_A && code <= LOWERCASE_Z
}

function caesarEncrypt({ plainText, key }: EncryptParameters): string {
    const k = normalizeKey(key)
    const inputArr = plainText.split("")
    inputArr.forEach((element, index) => {
        const code = element.charCodeAt(0)
        if (isLowercaseLetter(code)) {
            inputArr[index] = String.fromCharCode(wrapLowercaseCode(code + k))
        }
    })
    return inputArr.join("")
}
```

```
}

function caesarDecrypt({ encryptedText, key }: DecryptParameters): string {
  const k = normalizeKey(key)
  const inputArr = encryptedText.split("")
  inputArr.forEach((element, index) => {
    const code = element.charCodeAt(0)
    if (isLowercaseLetter(code)) {
      inputArr[index] = String.fromCharCode(wrapLowercaseCode(code, k))
    }
  })
  return inputArr.join("")
}

const encryptedString = caesarEncrypt({ plainText: "tasty", key: 5 })
console.log(encryptedString)
const decryptedText = caesarDecrypt({ encryptedText: encryptedString, key: 5 })
console.log(decryptedText)
```

Output:



Figure 1: Output of Caesar Cipher Implementation